

FACULTY OF TECHNOLOGY AND ENGINEERING**B.Tech. (CSE / CE / IT / AIML)**

Semester III | Academic Year 2026–27

CSUC201 – Fundamentals of Data Structure and Algorithms**PRACTICAL EXAMINATION**

Maximum Marks	50	Questions to Attempt	2
Q1	25 Marks	Q2	25 Marks

Instructions

1. Attempt ONE option from Question 1 and ONE option from Question 2.
2. Each question carries 25 marks.
3. The examination is completely programming-based. No theoretical answer, algorithm description, pseudocode, or written explanation is required.
4. Students must demonstrate the program using suitable normal, boundary, and exceptional test cases.
5. Appropriate functions must be created for major operations.
6. Built-in library functions that directly perform the required ordering, searching, or linked-list operations are not permitted.
7. For Question 2, use dynamically allocated nodes. STL list or similar ready-made linked containers are not permitted.
8. The student must identify and implement the most appropriate solution from the problem requirements.

No separate theoretical answer is required.

Question 1 – 25 Marks

Attempt Q1(A) OR Q1(B)

Q1(A) – Conference Entry Audit System

A technical conference records the badge ID of every person entering an exhibition hall. During a busy session, the same badge may appear several times because participants are allowed to leave and re-enter.

The organizers also need to perform several badge-verification requests after the event.

Develop a program that performs the following:

1. Read N badge IDs and preserve their original entry order.
2. Identify every badge ID that occurs more than once. For every such badge, display the badge ID and number of occurrences. Each repeated badge ID must appear only once in this report, and the report must follow the order in which the badge ID first appeared.
3. Create a second copy of the badge IDs arranged from smallest to largest.
4. Read Q badge IDs to be verified.
5. For every requested badge ID, use two different lookup implementations covered in the practical syllabus: one operating on the original sequence and one operating on the arranged sequence.
6. For each requested badge ID, display the required result and the number of comparisons made by both methods.
7. After all Q requests have been processed, automatically display the total comparisons by both methods and identify the better method for the given test data.
8. Correctly handle empty input, one badge, all IDs identical, no repeated IDs, requested ID absent, and first/last element requests.

Required output for each query:

```
Badge ID
Found / Not Found
First position in original record
Number of comparisons – Method 1
Number of comparisons – Method 2
```

Mark Distribution

Evaluation Component	Marks
Input handling and original data processing	3
Repeated-ID and frequency processing	4
Creation of correctly arranged data	5
Two lookup implementations	6
Comparison counting and automatic evaluation	4
Edge cases, modular functions, and program correctness	3
Total	25

Q1(B) – Inter-College Hackathon Result Processor

An inter-college hackathon stores Team_ID and Final_Score for every participating team. Team_ID is unique, while different teams may obtain the same score. The result system must generate the final ranking and answer result-related queries.

Develop a program that performs the following:

1. Read records of N teams.
2. Create the final ranking using these rules: higher score receives the better rank; if two teams have the same score, the smaller Team_ID appears first.
3. Display the complete ranked list as Rank, Team_ID, and Score.
4. Identify every score obtained by more than one team and display the score with the number of teams obtaining it.
5. Accept Q result queries. Each query contains a Team_ID. Display Team_ID, Score, and Final Rank, or display Team Not Found.
6. Implement two different ways of locating a requested team using ideas covered in the practical syllabus.
7. Count the number of record comparisons made by both implementations while processing all Q queries.
8. At the end, automatically display total comparisons for both methods and identify the better method for this input.
9. Correctly handle one team, equal scores, already ranked input, reverse-ranked input, highest/lowest scoring teams, and invalid Team_ID queries.

Mark Distribution

Evaluation Component	Marks
Record representation and input processing	3
Correct creation of final ranking	6
Duplicate-score frequency report	3
Query processing using two implementations	6
Comparison measurement and automatic evaluation	4
Edge cases, modularity, and correctness	3
Total	25

Question 2 – 25 Marks

Attempt Q2(A) OR Q2(B)

Q2(A) – Autonomous Greenhouse Inspection Network

A large automated greenhouse contains several inspection stations. A robot continuously visits the stations. Each station has a unique integer Station_ID. After the last station, the robot must automatically continue to the first station; it must also be able to travel in the opposite direction. Stations may be added or removed while the system is running, and no fixed maximum number of stations is known in advance.

Develop a dynamically allocated program supporting these commands:

ADD x – Add station x immediately after the robot's current station.

NEXT – Move the robot to the next station.

PREVIOUS – Move the robot to the previous station.

REMOVE – Remove the station at which the robot is currently located. If another station exists, move to the next available station; otherwise the network becomes empty.

JUMP k – Move the robot k stations forward. k may be very large (for example, 10000000), and the program should avoid unnecessary repetition of complete cycles.

DISPLAY – Starting from the current station, display exactly one complete round in the forward direction and exactly one complete round in the reverse direction. The program must not enter an infinite loop.

STATUS – Display the current station and total number of stations.

Additional Programming Requirement

After every ADD or REMOVE, execute a validation function that checks whether the entire network can still be traversed correctly in both directions. Display NETWORK VALID or NETWORK INVALID. The validation must be performed by the program.

- Handle an initially empty network.
- Handle one station only and removal of the only station.
- Handle repeated forward/reverse movement and very large k.
- Handle operations attempted on an empty network.

Mark Distribution

Evaluation Component	Marks
Dynamic node creation and initialization	4
Addition and removal operations	6
Forward and reverse movement	4
JUMP k handling and cyclic traversal	4
Network validation routine	4
Edge cases, modularity, memory handling, and correctness	3
Total	25

Q2(B) – Technical Support Ticket Navigator

A software company maintains active technical-support tickets dynamically. Every ticket contains Ticket_ID and Priority, where priority is between 1 and 5. The support engineer works with one ticket at a time and must be able to move both forward and backward through the active tickets.

New tickets must always follow this business rule: higher-priority tickets appear before lower-priority tickets; if two tickets have the same priority, the ticket received earlier remains first.

Develop a dynamically allocated program supporting these commands:

NEW id priority – Create a new ticket and place it in its correct position.

NEXT – Move the engineer to the next ticket, if one exists.

PREVIOUS – Move the engineer to the previous ticket, if one exists.

RESOLVE id – Remove the specified ticket from the active ticket collection.

CHANGE id newPriority – Change the priority of the specified ticket and reposition it so that all business rules remain satisfied.

SHOW – Display all active tickets from first to last as Ticket_ID and Priority.

SHOW_REVERSE – Display all active tickets from last to first.

CURRENT – Display the ticket currently selected by the engineer.

Additional Programming Requirement

Maintain a variable that counts the number of link modifications performed during every NEW, RESOLVE, and CHANGE operation. After each such operation, display the number of link modifications. At the end of execution, display the total number of link modifications.

The program must correctly handle:

- initially empty collection and only one ticket
- highest- and lowest-priority insertion
- several tickets with the same priority
- resolving the first, last, or current ticket
- changing a ticket from low to high priority and high to low priority
- invalid Ticket_ID
- movement beyond the first or last ticket

Mark Distribution

Evaluation Component	Marks
Dynamic ticket-node representation	3
Correct placement of newly created tickets	5
Removal and priority-change operations	7
Forward and backward navigation	4
Link-modification measurement	3
Edge cases, modularity, memory management, and correctness	3
Total	25

— End of Question Paper —